

SprintDesk — Sprint Management Dashboard

Role: Frontend Engineer

Submission: Public GitHub Repository + Live Deployment + Screen Recording

1. Assignment Overview

As part of our frontend engineering evaluation process, you are required to build **SprintDesk**, a sprint management dashboard designed for software development teams.

The assignment is intended to evaluate your ability to build a production-oriented React application with:

- Clean and scalable architecture
- Strong TypeScript practices
- Effective state management
- Server-state management
- Interactive drag-and-drop functionality
- Data visualization
- Reusable UI components
- Authentication and protected routes
- Responsive design
- Accessibility
- Performance optimization
- Testing
- Real-world problem solving

This is not intended to be a tutorial-style implementation. We are looking for the level of engineering quality, decision-making, and attention to detail that would be expected from a frontend engineer working on a production application.

2. Important Candidate Guidance

The assignment is intentionally challenging for the target experience level.

We value:

- Code Quality
- Good architectural decisions
- Clean and maintainable code
- Thoughtful prioritisation
- Proper handling of edge cases

- Responsive and accessible UI
- Clear documentation of technical decisions

A focused and well-executed implementation is preferable to a rushed implementation containing incomplete or unreliable features.

If you are unable to complete a requirement, document the limitation clearly in the README and explain what you would implement or improve with additional time.

3. Problem Statement

You are building **SprintDesk**, a single-page sprint management application for software teams.

The application should provide a central workspace where users can:

- Authenticate securely
- Manage sprint tasks through a Kanban board
- Drag and reorder tasks
- View task details
- Create and delete tasks
- Analyze sprint data
- Receive notifications
- Switch between light and dark themes
- Use a consistent component system

The application should behave like a real-world product rather than a static UI demonstration.

4. Technical Requirements

The following technology requirements are mandatory.

Area	Requirement
Framework	React 18+
Language	TypeScript with strict mode
Build Tool	Vite
Data Fetching	TanStack Query v5

Global State	Zustand
Styling	Tailwind CSS v3+
Routing	React Router v6+
Charts	Recharts or Visx
Drag & Drop	@dnd-kit/core
Testing	Vitest + React Testing Library
API	JSONPlaceholder + DummyJSON

Restrictions

The following are not permitted:

- Next.js
- Remix
- Create React App
- Angular
- Vue
- MUI
- Ant Design
- Chakra UI
- Shadcn UI
- Other external UI component libraries
- react-beautiful-dnd

Custom CSS may be used in addition to Tailwind CSS where necessary.

5. Functional Requirements

Task 01 — Authentication

Implement a complete authentication flow using the DummyJSON authentication API.

Required Features

- Login page with username and password
- POST authentication request to:

<https://dummyjson.com/auth/login>

- Store the access token in memory
- Store the refresh token using the specified local-storage simulation
- Implement an API interceptor that automatically attaches the Bearer token
- Simulate token expiration and implement silent token refresh
- Retry the failed request after successful token refresh
- Protect authenticated routes
- Redirect unauthenticated users to </login>
- Prevent authenticated users from accessing </login>
- Persist the user session after page refresh when the refresh token remains valid
- Logout functionality
- Clear authentication state during logout
- Redirect the user to </login> after logout
- Display a full-screen loading state while the initial session is being validated

Optional Bonus

- Remember Me functionality with simulated 30-day persistence
 - Password strength indicator
-

Task 02 — Kanban Sprint Board

Build a fully interactive Kanban sprint board with four columns:

1. Backlog
2. In Progress
3. Review
4. Done

Requirements

- Fetch the first 30 tasks from mock-data.json
- Use Zustand for board state
- Implement drag-and-drop using @dnd-kit/core
- Allow task reordering within and between columns
- Persist board state across page refreshes
- Display task priority, assignee, and due date
- Open task details in a side drawer
- Allow editing task details and adding comments
- Add new tasks with title, priority, assignee, and due date
- Delete tasks with confirmation
- Update column task counts dynamically

Optional Bonus

- Undo last drag-and-drop action

- Filter by priority or assignee
 - Keyboard-accessible drag-and-drop
-

Task 03 — Analytics & Data Visualisation

Create a dedicated Analytics page with responsive, interactive charts using real application/API data.

Include visualisations for:

- Sprint Velocity — number of completed tasks per sprint
- Task Status — distribution across board columns
- Priority Breakdown — task priorities across columns
- Completion Trend — task completion over time

Requirements

- Use Recharts or Visx
- Charts must be derived from actual board/API data, not hardcoded
- Analytics should update when board data changes
- Fully responsive, including 375px mobile viewport
- Include basic chart animations

Optional Bonus

- Custom date-range filtering
 - Export analytics as PNG
-

Task 04 — Design System & Component Library

Build a reusable component library from scratch using Tailwind CSS.

The library should include:

- Button
- Input
- Select / Dropdown
- Modal
- Toast
- DataTable
- Skeleton / Loading components

Components should be reusable, composable, responsive, and accessible, with appropriate states and interactions based on their purpose.

Optional Bonus

- Storybook implementation
 - Accessibility testing using axe-core
-

Task 05 — Real-Time Notification System

Implement a simulated real-time notification system using polling.

- Poll https://jsonplaceholder.typicode.com/posts?_limit=5
 - Treat new post IDs as new notifications
 - Add a notification bell with unread count
 - Display the latest 20 notifications with read/unread state. Support Pagination when more than 20 notifications are available.
 - Allow users to:
 - Mark notifications as read
 - Mark all as read
 - Persist notifications using Zustand + localStorage
 - Pause polling when the browser tab is hidden and resume when visible
 - Show a toast when new notifications arrive while the panel is closed
-

Task 06 — Performance, Accessibility & Testing

Focus on application quality, optimization, and reliability.

Required

- Lighthouse: Performance ≥ 88 and Accessibility ≥ 92
 - Accessibility: Keyboard-accessible interactions, proper form labels, and meaningful image alt text
 - Code Splitting: Use React.lazy + Suspense for route-level lazy loading
 - Optimization: Use React.memo, useMemo, and useCallback where appropriate
 - Testing: Unit tests for:
 - useToast
 - Zustand board store (add, move, delete)
 - Auth interceptor (refresh & retry)
 - All tests must pass with npm run test
-

06. Application Routes

The application must contain at least four distinct routes.

The recommended structure is:

- `/login`
- `/dashboard`
- `/board`
- `/analytics`

Authenticated routes must be protected.

07. Standards & Evaluation Criteria

The following requirements apply to the **entire application** and will be considered during evaluation in addition to the functional requirements listed above.

7.1 Scope Discipline — No Unnecessary Features

Candidates are expected to implement **only the features explicitly mentioned in this assignment**.

The implementation may include:

- Required features
- Clearly marked Bonus features

Candidates should **not add unrelated features, screens, workflows, libraries, or functionality** that are not part of the assignment.

Dead Code

The submitted repository must not contain unnecessary or unused code.

This includes:

- Unused components
- Unused hooks
- Unused utilities
- Unused imports
- Unused variables
- Commented-out old implementations
- Unused API functions
- Duplicate components

- Unused dependencies
- Abandoned experimental code

Dead code and unnecessary implementation will be considered negative during evaluation.

Candidates should remove unused code before submission.

7.2 State Management

The application must demonstrate **proper state management across the entire application**.

Candidates should identify the appropriate type of state and manage it using the correct approach.

At minimum, the implementation should clearly separate:

Server State

Server/API-related state should be handled using **TanStack Query v5**, including:

- API requests
- Loading states
- Error states
- Caching
- Refetching
- Polling
- Request lifecycle
- Query invalidation where appropriate

Client/Application State

Application-level client state should be handled using **Zustand** where appropriate, including:

- Authentication state
- Kanban board state
- Notifications
- Theme state
- UI/application state that needs to be shared across components

Local Component State

Component-level state should remain local when global state is unnecessary.

Candidates should avoid:

- Prop drilling through multiple unrelated components

- Duplicating the same state in multiple places
- Using Zustand for every piece of local UI state
- Using React Context as a replacement for proper application state management
- Storing server state unnecessarily inside Zustand

7.3 API Integration & Data Abstraction

The assignment is also intended to evaluate the candidate's **API integration architecture**, not only whether API requests work.

The provided mock data should be treated as a temporary data source that represents a real backend.

Candidates must avoid coupling UI components directly to the mock JSON structure.

Required Approach

API communication should be organized through a dedicated API/service/data-access layer.

For example:

UI Components

↓

Hooks / Query Layer

↓

API / Service Layer

↓

Data Source

↓

Mock JSON / Real API

The UI should not directly contain logic such as:

```
fetch('/mock-data.json')
```

throughout multiple components.

Instead, API access should be centralized so that replacing the mock data source with a real backend requires **minimal or no changes to the UI layer**.

Dynamic API Requirement

Candidates should implement the application using dynamic data access patterns.

The provided JSON data should simulate backend responses. The implementation should be structured so that the mock data source can later be replaced with real API endpoints.

7.4 Provided Mock Data

A `mock-data.json` file will be provided with the assignment.

Candidates are expected to use this data as the initial data source for the application wherever applicable.

The provided data may contain mock representations of:

- Users
- Authentication-related user data.
- Sprint tasks
- Task details
- Comments
- Notifications
- Analytics-related data
- Assignees
- Other data required by the application flows

Candidates must **not modify the provided mock data simply to make the UI work around an implementation issue.**

If the application requires additional derived data, the candidate should generate or transform it through the appropriate service/state layer.

Mock-data.json is the primary application data source for users, sprints, tasks, comments, and initial notification data. DummyJSON is used for authentication and token refresh. JSONPlaceholder is used only for simulated notification polling.

08. Submission Guidelines for the Assignment

8.1. GitHub Repository

Public GitHub repo with clean code and [README.md](#).

8.2. Architecture Document

Brief system architecture, components, technologies, and data flow.

8.3. API Documentation

API endpoints with request/response details. Swagger/OpenAPI preferred.

8.4. Screen Recording & Demo

Short video demonstrating the project, key features, APIs, and important implementation decisions.

8.5. Setup Instructions

Steps to run the project locally, including required environment variables.

8.6. Security

Do not commit passwords, API keys, or sensitive credentials.

8.7. Submission

Share all links/documents in one place and ensure they are accessible.